

EMS Project — Caching Implementation Study Guide

Spring Boot + JPA + Caffeine Cache | Practical implementation and interview/reference notes

1. Why Caching Is Important

Caching stores frequently requested data in a faster-access layer so that the application does not need to repeatedly execute the same expensive database operation. In the EMS project, employee data is a good candidate because employee records may be read frequently while changing less often than they are read.

Without caching: repeated GET requests can repeatedly execute Hibernate/JPA queries against MySQL.

With caching: after the first successful lookup, subsequent eligible requests can be served from the cache until the cached entry expires or is invalidated.

2. Why We Chose Caffeine

Option	Typical strength	Why it was not our first choice
Caffeine	Fast in-process/local cache; simple Spring integration	Best fit for our single-application EMS learning project
Redis	Distributed/shared cache; useful across application instances	Additional external service and operational complexity we do not currently need
Hazelcast	Distributed in-memory data grid/cache	More infrastructure and distributed-system concepts than required for this project

The decision was based on the current project scope: we wanted to learn and demonstrate Spring caching without introducing the additional infrastructure of a distributed cache. Caffeine is therefore a practical local-cache choice for this EMS application.

3. Caffeine Configuration

We enabled Spring's caching abstraction and configured a `CaffeineCacheManager` with two named caches: **employees** and **employeeList**.

```
@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {

        CaffeineCacheManager cacheManager =
            new CaffeineCacheManager("employees", "employeeList");

        cacheManager.setCaffeine(
            Caffeine.newBuilder()
                .maximumSize(500)
                .expireAfterWrite(10, TimeUnit.MINUTES)
                .recordStats()
        );

        return cacheManager;
    }
}
```

maximumSize(500): limits the configured cache to approximately 500 entries before Caffeine begins evicting entries according to its eviction policy. **expireAfterWrite(10 minutes):** an entry expires 10 minutes after it is written.

recordStats(): enables cache statistics such as hits and misses, which we observed through Spring Boot Actuator.

4. Cache Names and What They Store

employees stores an individual employee using the employee ID as the key.

employeeList stores the result of the operation that retrieves all employees. It is effectively a cache entry for the complete employee list.

5. Annotations Used in EmployeeServiceImpl

Annotation	Purpose in EMS
@Cacheable(value = "employees", key = "#employeeId")	Checks the employee cache first. On a miss, executes the method and stores the returned EmployeeDTO.
@Cacheable(value = "employeeList")	Caches the result of getAllEmployees(). The default key is used because the method has no parameters.
@CachePut(value = "employees", key = "#employeeId")	Always executes the update method and puts its returned EmployeeDTO into the employee cache.
@CacheEvict(value = "employeeList", allEntries = true)	Clears all entries in the employee-list cache after a create/update/delete operation so the list does not contain stale data.
@CacheEvict(value = "employees", key = "#employeeId")	Removes the individual employee entry after deletion.
@Caching(...)	Combines multiple cache operations on one method, such as updating an individual employee cache.

6. Final EmployeeService Cache Strategy

```
@CacheEvict(value = "employeeList", allEntries = true)
public EmployeeDTO createEmployee(EmployeeDTO employeeDTO) {
    ...
}

@Cacheable(value = "employees", key = "#employeeId")
public EmployeeDTO getEmployeeById(Long employeeId) {
    ...
}

@Cacheable(value = "employeeList")
public List getAllEmployees() {
    ...
}

@Caching(
    put = @CachePut(value = "employees", key = "#employeeId"),
    evict = @CacheEvict(value = "employeeList", allEntries = true)
)
public EmployeeDTO updateEmployee(Long employeeId, EmployeeDTO updatedEmployeeDTO) {
    ...
}

@Caching(
    evict = {
        @CacheEvict(value = "employees", key = "#employeeId"),
        @CacheEvict(value = "employeeList", allEntries = true)
    }
)
public void deleteEmployeeById(Long employeeId) {
    ...
}
```

7. Why Cache Eviction Is Important

Caching introduces a new concern: **stale data**. If employee 22 is cached and the database is subsequently updated, the old cached EmployeeDTO could otherwise continue to be returned.

That is why our write operations also manage the cache:

- Create → evict employeeList because the complete list has changed.
- Update → put the new individual employee value into employees and evict employeeList.
- Delete → evict the individual employee and employeeList.

8. How We Verified the Cache

We invoked the same get-employee endpoint twice. The first request executed the Hibernate SELECT, while the subsequent request could be served from the cache. We also exposed Spring Boot Actuator cache metrics and observed the **cache.gets** metric with hit/miss tags.

The Actuator response showed tags including **result = hit/miss**, **cache = employees**, and **name = employees**. This provided an observable way to verify cache behavior rather than relying only on application code.

9. Important Q&A;

Q1. Does JPA already provide caching?

Yes. JPA providers such as Hibernate can provide first-level caching through the persistence context and can also support a second-level cache. The Caffeine implementation we added is an application-level Spring Cache solution and is separate from Hibernate's persistence-context behavior.

Q2. Why not Redis?

Redis is an excellent choice when a shared/distributed cache is needed, especially when multiple application instances must see the same cached data. For our current single-application EMS project, adding Redis would introduce infrastructure that is not necessary to demonstrate the caching concepts we are learning.

Q3. Why not Hazelcast?

Hazelcast is also well suited to distributed in-memory caching and data-grid scenarios. Caffeine is simpler for our current local, single-JVM use case.

Q4. What happens on a @Cacheable cache hit?

Spring's caching abstraction can return the cached value without invoking the underlying method. Consequently, the database query inside that method is avoided.

Q5. What happens on a cache miss?

The method executes normally. Its returned value is then stored in the configured cache for subsequent lookups.

Q6. Why use @CachePut for update?

@CachePut does not skip the method. The update executes, and the returned value is placed into the cache. This lets the individual employee cache immediately contain the updated EmployeeDTO.

Q7. Why evict employeeList after create/update/delete?

Because the cached list represents a previous snapshot of all employees. Any create, update, or delete can make that snapshot stale.

Q8. Why use @Caching?

Because update and delete need multiple cache operations. @Caching groups those operations on the same service method.

Q9. What does recordStats() do?

It enables cache statistics collection, allowing us to inspect cache behavior such as hits and misses through the configured monitoring/Actuator integration.

Q10. Is Caffeine a distributed cache?

No. Caffeine is primarily an in-process/local cache. Each application instance has its own cache.

Q11. What is the main trade-off of our Caffeine approach?

It is simple and very fast for a single application instance, but cached data is not automatically shared between multiple application instances.

Q12. What is the most important thing to remember?

Caching improves read performance, but it creates a cache-consistency problem. Every write path must therefore have an intentional cache update or eviction strategy.

10. Practical Takeaway for the EMS Project

The caching implementation demonstrates several useful Spring Boot skills: Spring Cache abstraction, Caffeine configuration, cache-aside-style read caching, cache invalidation, cache replacement on updates, Actuator-based observability, and thoughtful selection of infrastructure based on application requirements.

This implementation is intentionally practical rather than deeply focused on the internal AOP/proxy mechanics. A deeper study of Spring's cache interceptor, proxies, key generation, cache resolution, concurrency behavior, and interaction with transactions can be taken up later.